PRISM

The short version

What PRISM does today, what we want it to become, how people will use it, and the order to build it. 17 pages, plain language, no code.

In one minute

Today: you sign in with GitHub, paste a repository, and PRISM reads the code and gives a health score, a 3D map, a list of risks (leaked keys, slow patterns, risky dependencies) and an AI summary.

The problem: every check is a one-off. Nothing tells you what *changed*, nothing runs on its own, and you cannot mark a finding as 'ignore' or turn it into a task.

The plan: make PRISM something a team uses every day: it runs on every code change, reports on the pull request, remembers what you decided, and lets you act on findings.

Contents

1 The idea on one page 2

2 The big picture 3

3 How it works today 4

4 Every feature in plain words 5

5 Nine workflows 8

6 How it fits together 13

7 The rules everything follows 14

8 Build plan 15

Exists today	New (planned)	Decision	Outside service	Warning / failure

Same material, in more detail: the 51-page PDF (`PRISM-complete.pdf`) and the Figma board in the Alpha team.

1 The idea on one page

Today	The gap	What we add
- Sign in with GitHub - Paste a public or private repo - Score, 3D map, risks, AI summary - History saved per commit	- Every check is manual - No 'what changed?' - Cannot ignore or track a finding - Nothing happens on a pull request	- Ignore / accept findings - Compare two runs, score trend - See the code next to each finding - Automatic checks on every push and pull request - Alerts, badge, GitHub issues - Exports, teams, plans, more languages, AI chat

Four releases

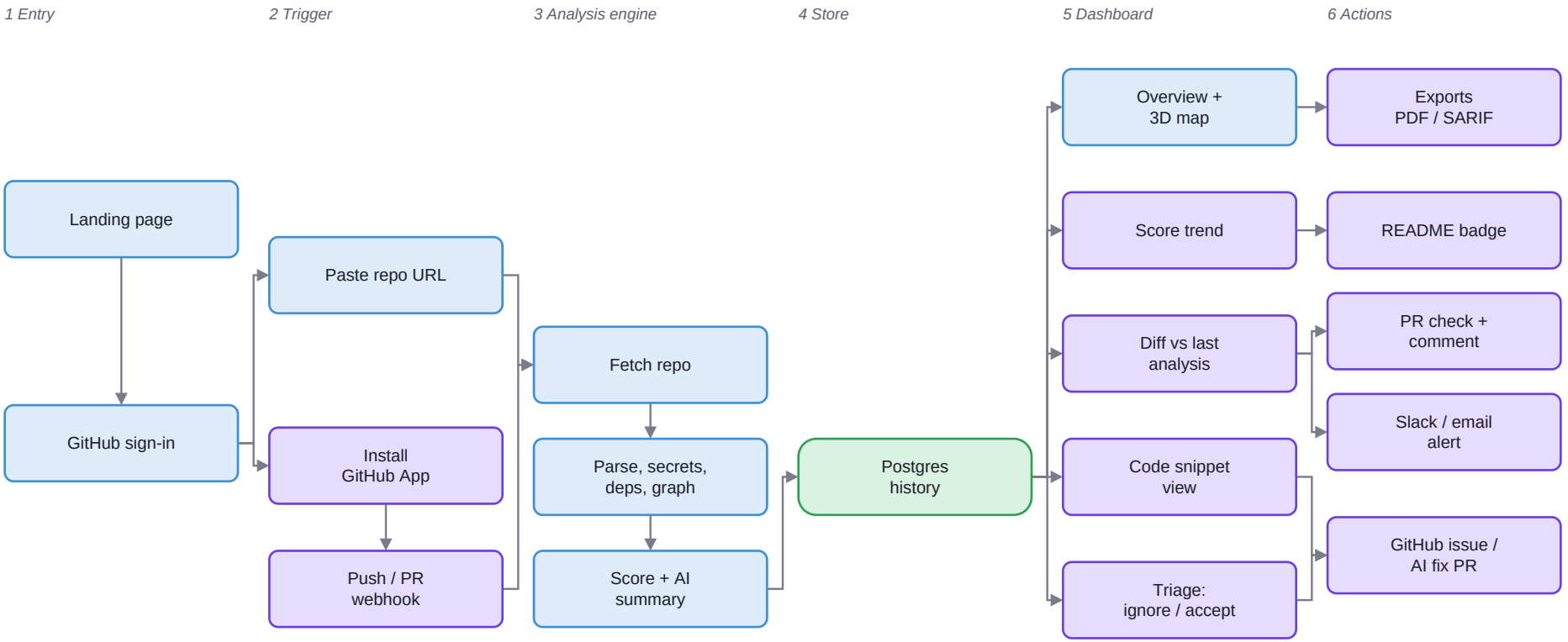
Release	Milestones	What people get	Needs new setup?
1 Actionable	M0 to M3	Ignore findings, compare runs, score trend, code snippets	No. Stays inside the app.
2 Continuous	M4 to M7	Runs on every push, pull request checks and comments, alerts, README badge, GitHub issues	Yes: GitHub App, background jobs
3 Insight	M8 to M10	Exports, hotspots, compare repos, one dashboard for all repos	No
4 Product	M11 to M15	Plans and billing, teams, share links, more languages, AI chat and AI fix pull requests	Yes: Stripe, team data

How to read the rest

- **Section 2** is the big picture on one diagram.
- **Section 3** explains how PRISM works today and the problems we must fix first.
- **Section 4** lists every feature in plain words, grouped by release.
- **Section 5** shows nine workflows: what the person does and what PRISM does.
- **Sections 6 and 7** cover how it fits together and the rules everything follows.
- **Section 8** is the build plan: order, effort, risks and decisions.

2 The big picture

Read left to right: how people arrive, what starts an analysis, what PRISM does, where results are kept, what people see and what they can do next.



3 How it works today

Step	What PRISM does	Limit
1 Download	Fetches the repository's main branch from GitHub	150 MB, 6,000 files
2 Read the code	Parses JavaScript, TypeScript and Python; measures how complex each function is; flags risky patterns	Only those 3 languages
3 Find leaked secrets	Looks for committed keys and passwords; always hides the value	Pattern based, can be wrong
4 Map dependencies	Works out which file imports which; finds circular imports	Draws the top 700 files
5 Check libraries	Compares your dependencies with a public list of known vulnerabilities (OSV.dev)	Needs the internet
6 Score and explain	Health score out of 100, risk per file, AI summary using facts only (never the source code)	Issues capped at 300

Problems we found in the current code (and fix first)

Problem	Why it matters	Fix
A problem's ID includes its line number	Edit the file and the same problem looks 'new' and 'fixed'. Compare and ignore would be wrong.	Stable ID that ignores line numbers (M0)
Only 7 characters of the commit ID are saved	Cannot reliably fetch the code at that commit or link to it	Save the full ID (M0)
Lists are cut at 300 issues	A fixed issue can make hidden ones appear as 'new'	Say so when a compare may be approximate (M2)
Private results are reused for 30 minutes	A new commit could show an old result	Reuse only for the same commit (M4)
Database setup runs on first use	Cannot make safe later changes	Numbered, repeatable migrations (M0)
No tests at all	Every change risks breaking a rule or the score	Tests and automatic checks (M0)

4 Every feature in plain words

Release 1 - foundations (Phase 0)

Feature	What it does	Key rule	Done when
F0.1 Safe database changes	Every database change is a numbered, repeatable step.	Applied once, all or nothing.	A fresh database matches today's.
F0.2 Stable problem IDs	Each problem keeps its ID even when code moves.	Built from rule + file + function + line text, never the line number.	Moving code does not change IDs.
F0.3 Score in one place	The health score is calculated by one function and its inputs are saved.	Lets us recalculate with ignored issues removed.	Same repo gives the same score as today.
F0.4 Tests and automatic checks	Checks run on every change.	Lint, types, tests, build; tests never use the network.	Every rule has a pass and a fail test.

Release 1 - actionable (Phase 1)

Feature	What it does	Key rule	Done when
F1.1 Ignore findings	Mark a problem ignored, false positive or accepted risk.	Per person. Accepted risk can expire. Score recalculates.	Ignoring a critical raises the score by 8.
F1.2 Compare two runs	Shows new, fixed and unchanged problems and the score change.	Matched by stable ID; ignored ones are left out.	Fix one and add one gives 1 fixed, 1 new.
F1.3 Score trend	A chart of the score over time.	Shows the adjusted score; a switch shows the raw one.	Chart matches its table.
F1.4 Code snippets	Shows the exact lines next to a finding.	Fetched at that commit; secret values hidden.	Highlighted line is the finding's line.

Release 2 - continuous (Phase 2)

Feature	What it does	Key rule	Done when
F2.1 Connect GitHub	Install PRISM on chosen repos.	Read-only code access. Write access only later, for fix PRs.	A push creates a job within 5 seconds.
F2.2 Background analysis	Analyses every push and pull request by itself.	Signed request, a queue, retries after 1, 5, 15 minutes.	A forged request creates nothing.
F2.3 Pull request check and comment	Reports what the change did to health, on the pull request.	Only new problems count. One comment, updated each push.	Adding eval fails; removing it passes.
F2.4 Repo settings	How strict the check is, per repo.	Default: fail on a new critical or secret; preview on the last 10 PRs.	Minimum score 90 fails a score-85 PR.
F2.5 README badge	A score badge for your README.	Public repos open; private repos need a secret link.	Badge equals the dashboard.
F2.6 Alerts	Slack or email when something important happens.	New critical, new secret, big score drop. At most one per repo per hour.	A committed key reaches Slack in 2 minutes.
F2.7 GitHub issue from a finding	One click creates a tracked issue.	Never creates a duplicate; secrets never copied.	Clicking twice makes one issue.

Release 3 - insight (Phase 3)

Feature	What it does	Key rule	Done when
F3.1 Exports and SARIF	Markdown, JSON, CSV, print to PDF, and SARIF for GitHub's Security tab.	Secret values always hidden.	SARIF passes GitHub's format check.
F3.2 Hotspots by change	Finds files that are complex and also change often.	One GitHub query for the top 30 files; the health score is unchanged.	Complex and busy beats complex and idle.
F3.3 Compare repos	Up to 4 repos side by side.	Uses each repo's latest result.	Numbers equal each dashboard.
F3.4 All-repos dashboard	One page showing which repos need attention.	Latest result per repo; biggest score drops first.	Totals equal the sum of rows.

Release 4 - product (Phase 4)

Feature	What it does	Key rule	Done when
F4.1 More languages	Go, Java, Rust and a proper Python reader.	Loaded only when needed.	Python change moves scores by 2 points or less.
F4.2 Ask your repo	Chat that answers with links to findings.	Facts only, read-only tools, code only if you click.	Every answer cites a real finding.
F4.3 AI fix pull requests	Proposes a fix and opens a pull request.	Opt-in. Patch must apply and stop the rule. Never merges.	A bad patch is never offered.
F4.4 Plans and billing	Free, Pro and Team through Stripe.	One function decides every limit. Nothing is deleted on downgrade.	Replaying a payment event changes nothing.
F4.5 Teams	Share repos and settings with a team.	Owner, admin, viewer.	A viewer cannot change settings.
F4.6 Share links	A read-only link for people without an account.	24 hours, 7 or 30 days; can be revoked.	Revoked links stop within a minute.
F4.7 Settings and first-run help	Settings, a 3-step first run, clear error screens.	Failed runs show a Retry button.	New user reaches a first result in 3 minutes.

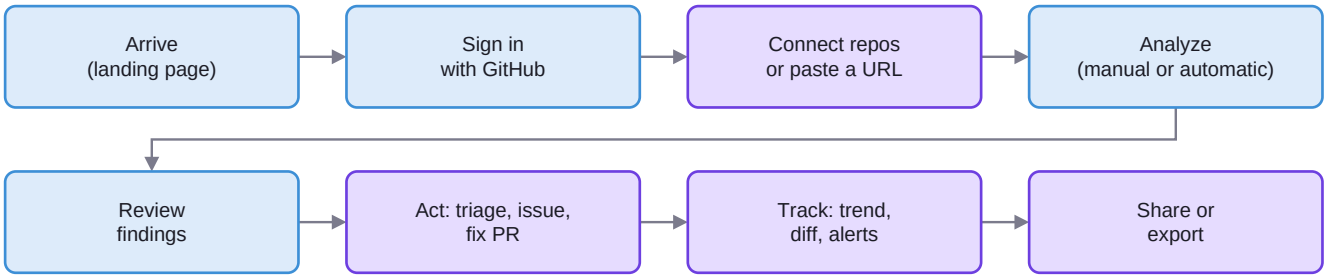
The three rules behind most features

- **Only new problems matter.** Checks and alerts react to what a change introduced, not to the old backlog.
- **Your decisions are remembered.** Ignore something once and it stays ignored, even when the line moves.
- **Never surprise people.** Secrets are always hidden, the AI sees facts (not code) unless you click, and nothing is auto-merged.

5 Nine workflows

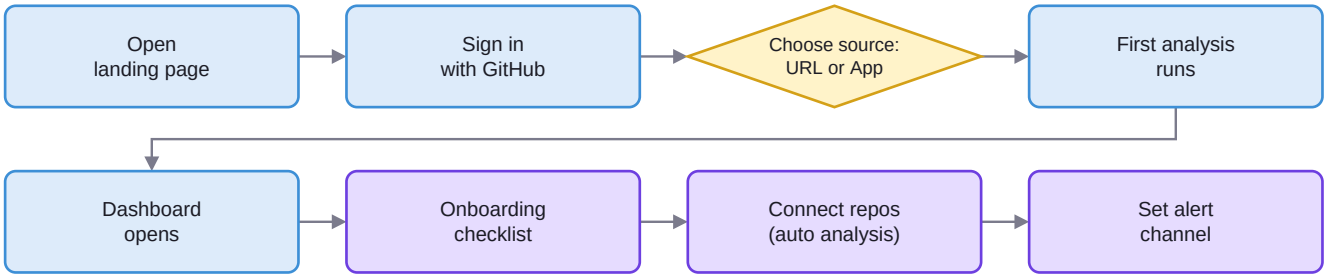
What the person does and what PRISM does. The diagram shows the main path; the notes cover what goes wrong.

The whole journey



W1 First-time setup

Who: New user. **When:** First visit.

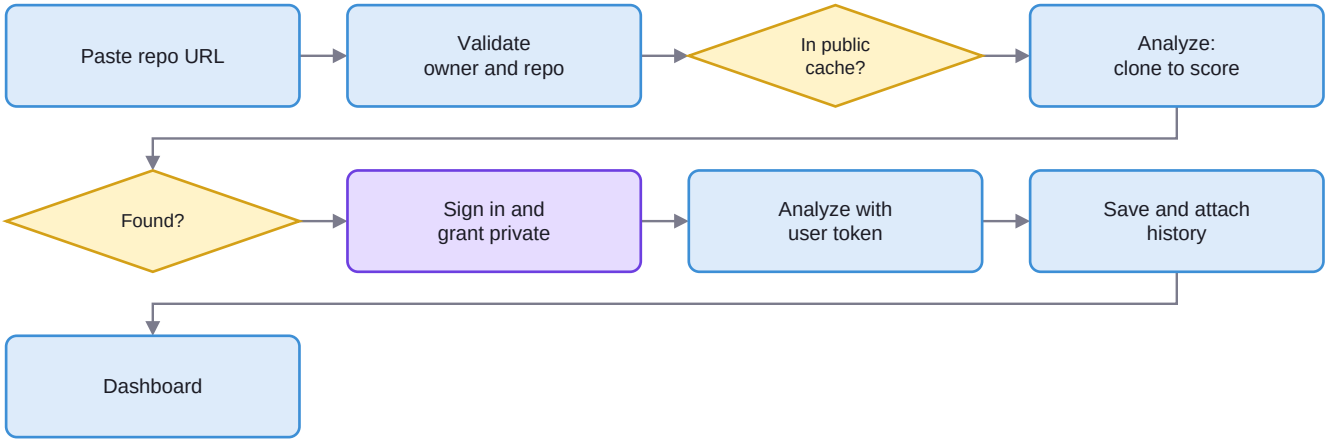


1. Sign in with GitHub (public profile only).
2. Paste a repository or install the GitHub App.
3. The first analysis runs with a progress screen.
4. The dashboard opens: score, 3D map, top risks, AI summary.
5. A checklist suggests connecting repos, ignoring noise and adding an alert.

Watch for: Sign-in cancelled: back to setup. Repo not found: sign in or check the name.

W2 Analyze a repository manually

Who: Any user. **When:** Any time.

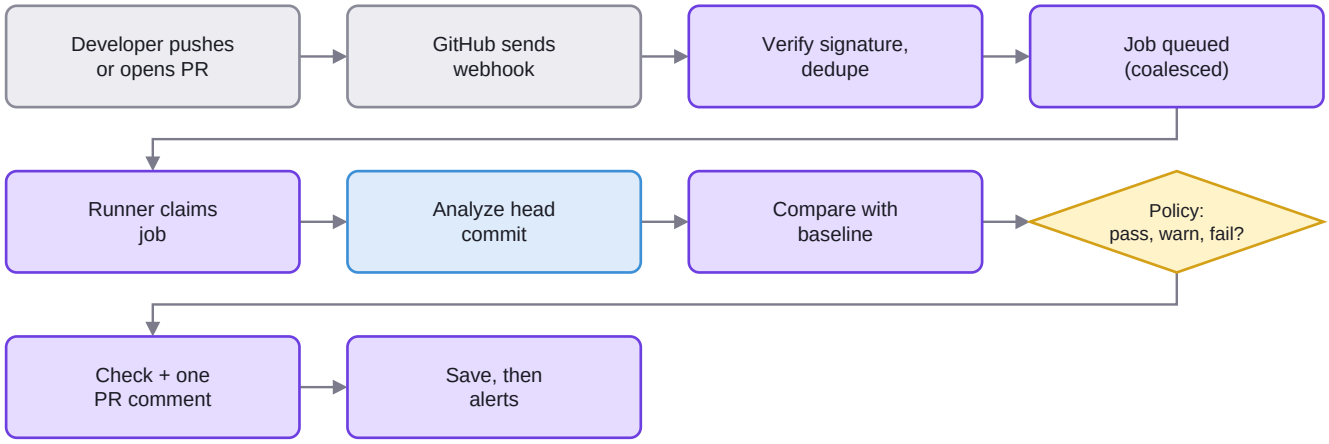


1. Paste owner/repo. A bad name gets an immediate, friendly error.
2. A public repo seen recently opens instantly from the shared cache.
3. Otherwise PRISM downloads and analyses it.
4. If GitHub says 'not found' it may be private: sign in and allow private access.
5. Private repos use your own token and are never shared; the result is saved with its trend.

Watch for: Too large, or no supported files: PRISM says what was skipped and why.

W3 Automatic analysis on push and pull request

Who: Developer (GitHub). **When:** Every push to the default branch and every pull request update.

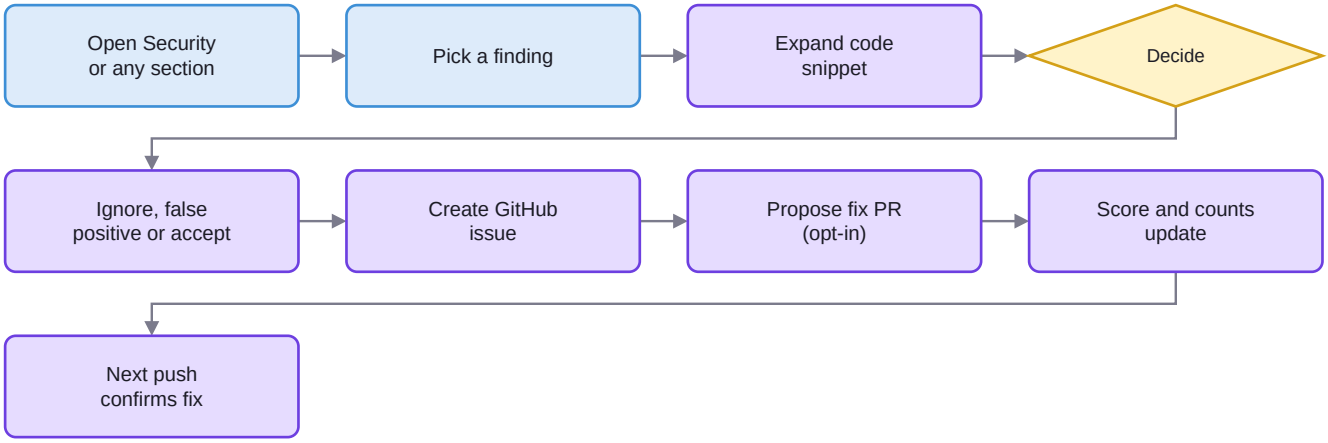


1. A developer pushes or updates a pull request; GitHub sends a signed message.
2. PRISM checks the signature, ignores repeats and queues a job (old queued jobs for the branch are dropped).
3. A runner analyses the new commit and compares it with the base commit.
4. Rules decide: new secret or critical fails, new high warns, otherwise passes.
5. One check and one comment appear on the pull request; then the result is saved and alerts go out.

Watch for: Fork pull requests PRISM cannot read get a neutral check. Missing baseline: noted in the comment.

W4 Review and act on a finding

Who: Developer. **When:** After an analysis, from any issue list.

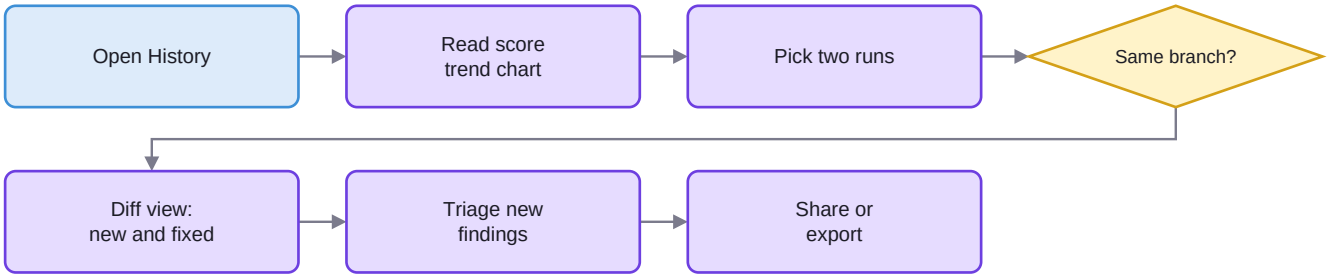


1. Open a finding and expand the code around it.
2. Read the AI explanation and its 'might be a false alarm' rating.
3. Choose: set aside (ignore, false positive, accept for a while), track (GitHub issue), or fix (AI pull request, opt-in).
4. The score and counts update immediately.
5. On the next push PRISM marks the finding fixed if it is gone.

Watch for: Signed out: these buttons are hidden. Leaked keys never get an AI fix: rotate the key.

W5 History, trend and diff

Who: Developer or lead. **When:** To see what changed between two runs.

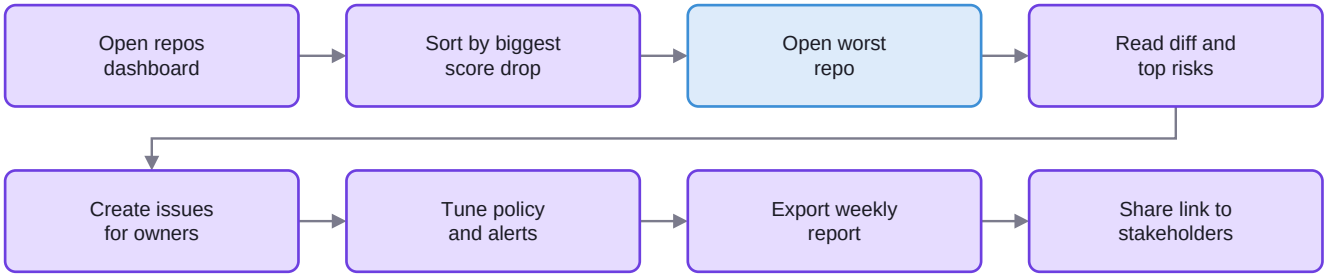


1. Open History: a chart of the score and one row per run.
2. Pick two runs (default: latest against the previous one on the same branch).
3. See new, fixed, changed and unchanged problems and the score change.
4. Ignore new findings right there, or export the comparison.

Watch for: First run: nothing to compare yet. Old runs without stable IDs are compared approximately and labelled so.

W6 Team lead weekly review

Who: Team lead or maintainer. **When:** Weekly.

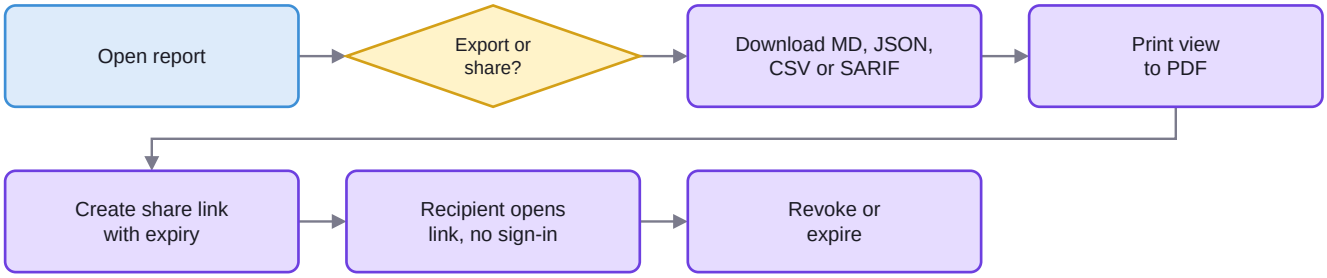


1. Open the all-repos dashboard; sort by biggest score drop.
2. Open the worst repo; read the comparison with last week.
3. Create GitHub issues for the findings that need an owner.
4. Adjust the repo's strictness and alerts using the preview.
5. Export a report or share a link with people outside PRISM.

Watch for: Repos not analysed for 30 days or with failing runs are highlighted.

W7 Export and share a report

Who: Any signed-in user. **When:** When results must leave PRISM.

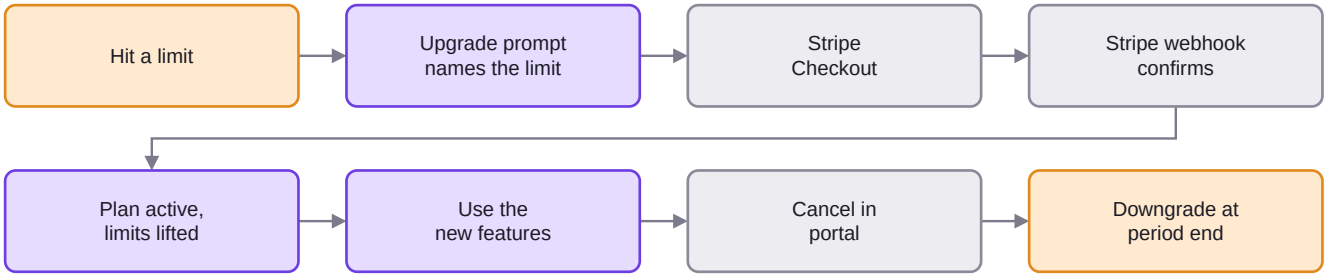


1. Open a report and press Export or Share.
2. Export: Markdown, JSON, CSV or SARIF; or print to PDF.
3. Share: choose 24 hours, 7 or 30 days; a warning appears for private repos.
4. The recipient opens the link without an account and sees a read-only snapshot.
5. You can revoke it at any time.

Watch for: Free plan: share links limited, with an upgrade prompt. Secrets are hidden everywhere.

W8 Plan limits and billing

Who: User or team owner. **When:** When a limit is reached or a plan changes.

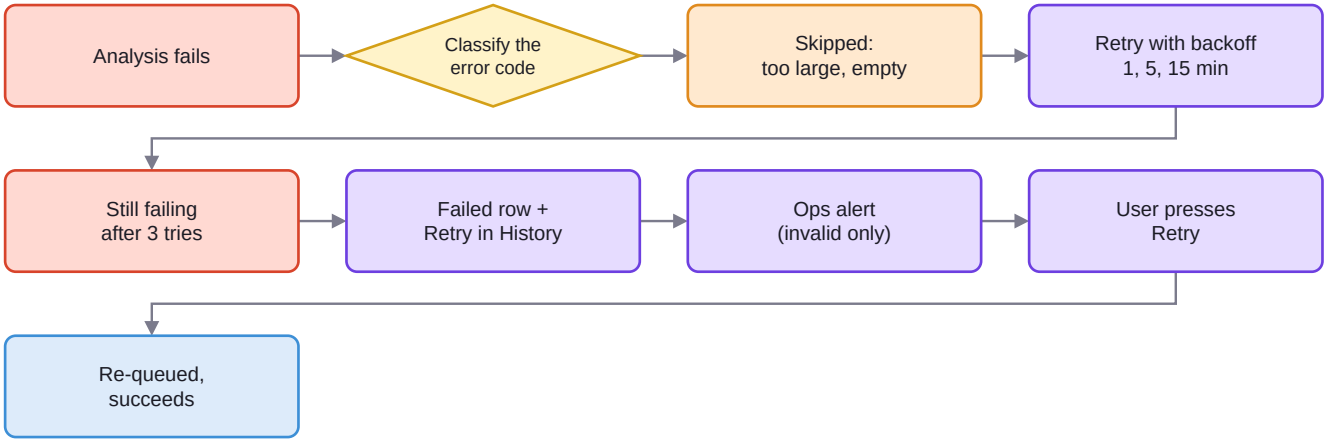


1. You hit a limit (for example a fourth private repo).
2. PRISM says which limit and what each plan includes.
3. You pay on Stripe's page; Stripe tells PRISM, which switches the plan on at once.
4. If you cancel, you drop to Free at period end: automatic analysis pauses, nothing is deleted.

Watch for: Payment fails: the plan stays for 7 days with a banner.

W9 Failure and recovery

Who: System, then user. **When:** When an analysis cannot complete.

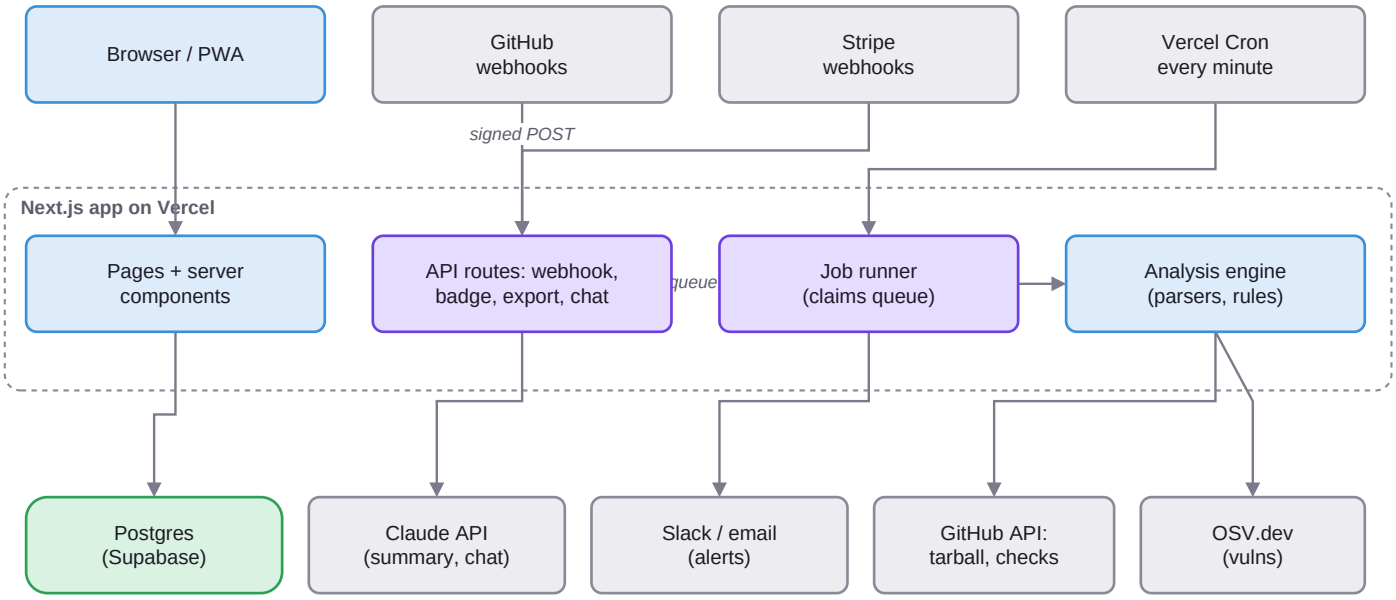


1. An analysis fails (GitHub unreachable, rate limit, too large, unexpected).
2. Too large or empty: marked skipped with the reason.
3. Network or rate limit: retried after 1, 5 and 15 minutes.
4. After the third failure it shows as a failed run in History with a Retry button.
5. Unexpected errors also alert the person running PRISM.

Watch for: Page views show the same message immediately instead of waiting for retries.

6 How it fits together

Everything stays inside your existing app. The only new moving part is a background runner started every minute that works through a queue kept in the database.



New database tables, in plain words

Table	What it holds
issue states	Your decisions: ignored, false positive, accepted (with expiry), and the GitHub issue link
repo settings	How strict each repo's check is, alert rules, badge on/off
installations, jobs, deliveries	Which repos are connected, the queue of analysis jobs, and a record of messages already received
alert channels and log	Where alerts go (encrypted) and what was already sent
shares	Read-only report links with expiry and revoke
workspaces and members	Teams, roles and who owns what
subscriptions and usage	Plans, payment status and monthly counters
analyses (existing, extended)	Adds full commit ID, branch, how it was triggered, and the score inputs

7 The rules everything follows

Stable problem ID

Each problem gets an ID from its rule, file, function and the text of the line, never its line number. That is what lets PRISM say 'this is the same problem as last week' even after code moves.

The health score

Starts at 100 and loses points: 8 per critical issue (up to 40), then smaller, capped amounts for high and medium issues, complexity, copy-pasted code and circular imports. Because the formula lives in one place, ignored issues can be removed and the score recalculated.

What an ignored finding does

State	Shown in lists	Counts toward score and checks
Open	Yes	Yes
Ignored / false positive	Under 'Triaged'	No
Accepted risk	Under 'Triaged', with expiry date	No, until it expires
Fixed (automatic)	Shown in the comparison as fixed	n/a

How a pull request check decides (first match wins)

#	If...	Result
1	A new secret was added	Fail
2	A new critical issue was added	Fail
3	The score is below the minimum you set	Fail
4	The score dropped by more than you allowed	Fail
5	A new high issue was added	Warning
6	There is no earlier result to compare with	Neutral, and says so
7	Otherwise	Pass

Old problems never fail a check. That is what makes PRISM usable on an old codebase.

Safety basics for anything new

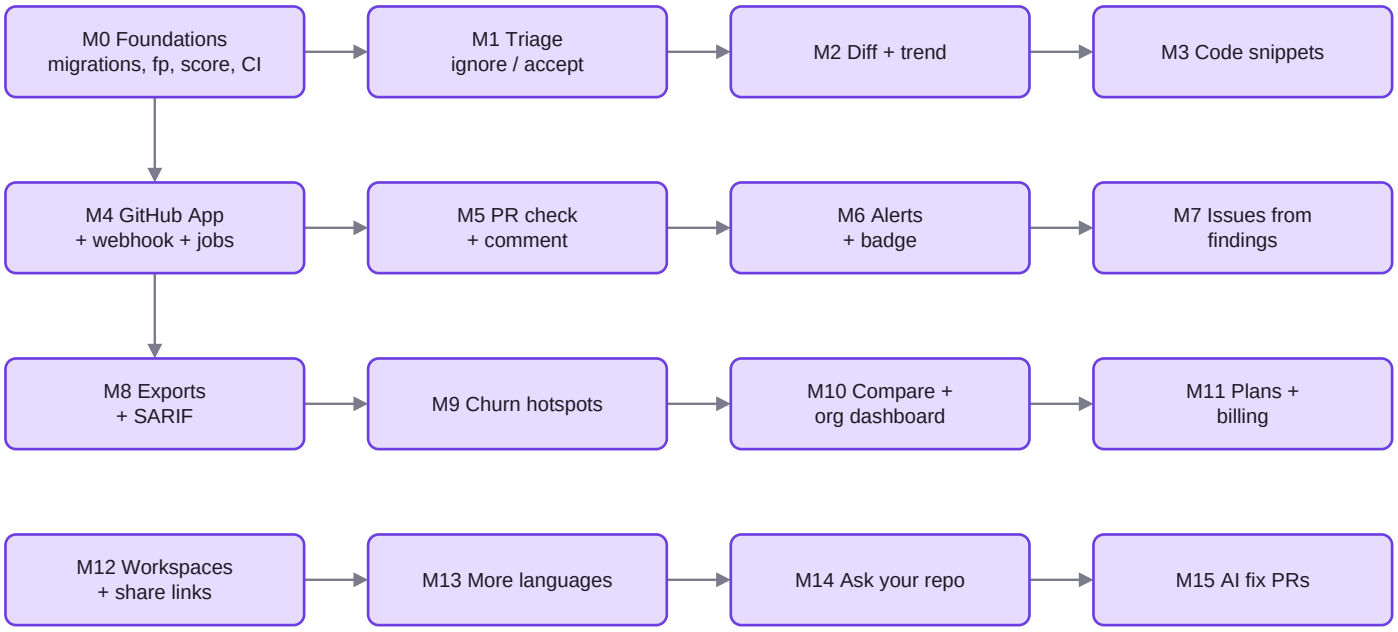
- Incoming GitHub messages are checked with a secret signature; every query is limited to the signed-in user's or team's data.
- Slack links are stored encrypted; public pages (badge, share links) are rate limited and hidden from search engines.
- Deleting an account removes everything, including shares, alerts and billing.

8 Build plan

Effort is a rough size for one developer: **S** 3 to 5 days, **M** 1 to 2 weeks, **L** 2 to 4 weeks. It is for ordering the work, not a promise.

#	Milestone	What you get	Effort	Needs
M0	Foundations	Safe database changes, stable IDs, score in one place, tests	L	-
M1	Ignore findings	Triage with notes and expiry; score recalculates	M	M0
M2	Compare and trend	Compare two runs; score chart	M	M0, M1
M3	Code snippets	Show the lines for each finding	S	M0
M4	GitHub App and jobs	Install on repos; analyse on every push in the background	L	M0
M5	Pull request check	Check and one comment per pull request; repo settings	L	M2, M4
M6	Alerts and badge	Slack/email alerts; README badge	M	M4
M7	GitHub issues	One-click issue from a finding	S	M4
M8	Exports and SARIF	Download formats; GitHub Security tab	M	M0
M9	Hotspots by change	Complex and busy files first	S	M0
M10	Compare and all-repos	Compare repos; one dashboard	M	M2
M11	Plans and billing	Limits in one place; Stripe	L	M5
M12	Teams and share links	Workspaces, roles, read-only links	L	M11
M13	More languages	Go, Java, Rust, proper Python	L	M0
M14	Ask your repo	AI chat with citations	M	M2
M15	AI fix pull requests	Proposed fix as a reviewable PR	M	M7, M14

The order and why



Order is the recommended sequence. M2 must land before M5 (PR comments reuse the diff); M4 before everything automatic.

Critical path: M0, M1, M2, M4, M5, M11. Everything else can move without delaying the paid plan.

Why milestones 0 to 3 come first

- **Everything else depends on M0.** Stable IDs, saved score inputs, safe database changes and tests are needed by almost every later feature.
- **No new setup.** They use only what you already have (database, GitHub sign-in, the analysis engine), so the first release cannot break anything outside the app.
- **The data already exists.** Every analysis is saved, so ignoring, comparing and the trend only read it.
- **They fix weak spots** found in the code (line-number IDs, short commit IDs, no tests).
- **They answer the main problem:** a reason to come back.
- **Pull request comments (M5) reuse the compare feature (M2).** Building them first would show false 'new problem' noise.

Risks and how the plan handles them

Risk	How it is handled
GitHub rate limits	Each connected repo has its own allowance; two calls per analysis; repeated pushes are merged into one job; retries wait 1, 5, 15 minutes.
Hosting time limits	Each analysis has a 90-second budget; very large repos are skipped with a clear reason.
Problem IDs that drift	Built and tested in M0 before anything depends on them.
Duplicate or forged webhooks	Signature check, repeat detection, and one comment per pull request.
Noisy findings	Ignore and false-positive options; only new problems fail a check; AI false-alarm rating.
AI cost and privacy	Plan quotas; facts only; code only after an explicit click, with secrets hidden.

Plans (a proposal, to be decided)

	Free	Pro	Team
Private repos	3	Unlimited	Unlimited
Automatic analysis	No	10 repos	Unlimited
Pull request checks and alerts	No	Yes	Yes
History kept	30 days	1 year	2 years
AI chat per day	10	100	300 per seat
AI fix pull requests per month	0	20	100

Decisions made

Decision	Why
Problem IDs ignore line numbers	So compare and ignore survive edits
Your ignore decisions apply when results are read, not inside the shared analysis	Keeps the public cache neutral for everyone
Ignored findings change the score, with a switch to see the raw score	Honest trends without a sudden jump
Old problems never fail a pull request check	So PRISM is usable on existing code
Background work uses a database queue run by a timer	No new servers needed
GitHub App for repo access; keep the current sign-in	Access that does not depend on one person's login
Ask for code-write permission only for AI fix pull requests	Smallest permission first
PDF export by printing a report page	No extra cost or library

Questions for you

- **Order:** start with M0 to M3 (everything inside the app) or go straight to pull request comments (M4, M5)? I recommend M0 to M3.
- **Free plan:** are 3 private repositories and no automatic analysis the right limits?
- **Alerts:** Slack first, email first, or both?
- **AI and code:** is it acceptable that a snippet is sent to the AI only when someone clicks for chat or fix pull requests?
- **Sign-in:** keep the current GitHub sign-in and add a GitHub App, or merge into one app?
- **Languages:** after JavaScript, TypeScript and Python, which first: Go, Java or Rust?